

Pruebas unitarias frontend

Margie Nicol Matamoros Buitrago & Julian Steban Ruiz Triviño

Universidad de Cundinamarca

Ingeniería de software

German Ortiz Diaz

2026

Contenido

Pruebas unitarias frontend.....	4
Figura 1	5
Figura 2	5
Figura 3	6
Nota: se ve el resultado de las pruebas de login Fuente: propia (2026) Figura 4.....	6
Figura 6	7
Figura 9	8
Figura 10	8

Lista de Figuras

Figura 1	5
----------------	---

Pruebas unitarias frontend

En este documento se detallan las pruebas unitarias desarrolladas para el *frontend* de cada una de las interfaces de “Mi Hogar 360”. Lo que se hizo para estas pruebas fue aislar el código de cada componente de sus dependencias externas. Utilizando el entorno de pruebas de Angular (TestBed) junto con el *framework* Jasmine, se implementaron simulaciones (*Mocks* y *Spies*) para servicios como la autenticación y el enrutador nativo (*provideRouter*). Esto permitió validar el comportamiento individual de la interfaz sin ejecutar peticiones reales al *backend* ni realizar saltos de página en el navegador.

De manera específica, la estrategia de pruebas abarcó los siguientes aspectos funcionales:

Validación de Formularios: Comprobación estricta de campos obligatorios, formatos correctos (ej. correos electrónicos), longitudes mínimas y validadores personalizados (ej. coincidencia de contraseñas cruzadas en el registro).

Interacción y Estado de la Interfaz (DOM): Verificación del correcto funcionamiento de elementos dinámicos, como la apertura y cierre de ventanas emergentes (modales), el bloqueo del *scroll*, la visibilidad de contraseñas y la aplicación de clases CSS condicionales (ej. cambio a modo oscuro o marcado de tareas completadas).

Lógica de Negocio y Filtrado: Evaluación de las funciones internas del controlador, asegurando que los arreglos de datos reaccionen bien a la búsqueda de inventario, el filtrado de tareas por estado (pendiente/proceso/completado) y los límites matemáticos (ej. no permitir menos de 1 residente en la configuración).

Procesos Asíncronos: Uso de herramientas de simulación de tiempo (*fakeAsync* y *tick*) para garantizar que la interfaz maneje correctamente los estados de carga (*loading*) mientras “espera” la respuesta simulada del servidor antes de redirigir al usuario.

Prevención de Conflictos Externos: Creación de objetos globales simulados para *scripts* de terceros (como el botón de Google Sign-In) garantizando que el entorno de pruebas no colapse por dependencias ajenas al código propio.

A continuación, se presentan las capturas de pantalla con la ejecución y los resultados arrojados por el motor Karma, evidenciando que todos los escenarios evaluados se completaron de manera exitosa y que la aplicación maneja el flujo del usuario de manera predecible y segura.

Figura 1

Pruebas unitarias compras

```
ComprasComponent
• no debería agregar un producto vacío
• debería incrementar y decrementar la cantidad de un producto
• debería agregar un producto a la lista si el texto no está vacío
• debería crear el componente correctamente
• no debería decrementar la cantidad por debajo de 1
• debería eliminar un producto de la lista
• debería alternar el estado de comprado del producto
```

Nota: se ve el resultado de las pruebas de compras Fuente: propia (2026)

Figura 2

Pruebas unitarias welcome

```
WelcomeComponent
• debería contener el título de Hogar360
• debería contener botones para iniciar sesión y crear cuenta
• debería crear el componente correctamente
```

Nota: se ve el resultado de las pruebas de welcome Fuente: propia (2026)

Figura 3*Pruebas unitarias login*

```

LoginComponent
  • debería cambiar la visibilidad de la contraseña
  • debería aceptar un formulario válido
  • el correo debería ser obligatorio
  • la contraseña debería ser obligatoria
  • debería aceptar un correo válido
  • debería crear el formulario de login
  • el correo debería tener un formato válido
  • debería crear el componente

```

Nota: se ve el resultado de las pruebas de login Fuente: propia (2026)

Figura 4*Pruebas unitarias perfil*

```

PerfilComponent
  • debería alternar el tema oscuro (Dark Mode)
  • debería crear el componente correctamente
  • debería cerrar el modal y limpiar el miembro seleccionado
  • debería abrir el modal con el miembro seleccionado
  • debería alternar el estado de completado de una tarea del miembro

```

Figura 5*Pruebas unitarias configuración*

```

ConfiguracionInicialComponent
  • debería crear el componente e iniciar en el paso 1

Paso 3: Zonas y Finalización
  • debería finalizar, mostrar carga y navegar al menú principal simulando el tiempo
  • no debería finalizar si no hay zonas seleccionadas
  • debería seleccionar y deseleccionar zonas

Paso 1: Datos Básicos
  • debería aumentar y disminuir el número de residentes respetando los límites
  • no debería avanzar al paso 2 si el nombre del hogar está vacío
  • debería avanzar al paso 2 si el nombre es válido
  • debería cambiar el tipo de vivienda

Paso 2: Invitados
  • debería agregar y eliminar campos de invitados
  • debería avanzar al paso 3 si los correos son válidos (o están vacíos)
  • no debería avanzar al paso 3 si hay correos inválidos

```

Nota: se ve el resultado de las pruebas de configuracion Fuente: propia (2026)

Figura 6

Pruebas unitarias inventario

```
InventarioComponent
• debería crear el componente correctamente
• debería filtrar los productos por categoría (ej. alimentos)
• debería filtrar los productos por coincidencia de texto en la barra de búsqueda
• debería devolver un arreglo vacío si la categoría no tiene productos
• debería registrar interacciones en consola / alertas
• debería combinar correctamente el filtro de categoría y texto de búsqueda
• debería inicializar mostrando todos los productos
```

Nota: se ve el resultado de las pruebas de inventario Fuente: propia (2026)

Figura 7

Pruebas unitarias estadística

```
EstadisticasComponent
• debería cambiar el periodo correctamente a anual
• debería iniciar con el periodo "mensual" por defecto
• debería contener los datos base del comparativo (3 meses de ejemplo)
• debería cambiar el periodo correctamente a trimestral
• debería crear el componente correctamente
• debería exportar PDF (simulación)
```

Nota: se ve el resultado de las pruebas de estadística Fuente: propia (2026)

Figura 8

Pruebas unitarias registrar

```
RegisterComponent
• debería inicializar el formulario con 4 campos vacíos
• debería crear el componente correctamente
• debería llamar a authService.register y navegar al dashboard si el formulario es válido
• debería cambiar la visibilidad de la contraseña
• debería marcar el formulario como inválido si está vacío
• debería no llamar al servicio si el formulario es inválido al hacer submit
• debería validar que el correo tenga formato correcto
• debería validar que la contraseña tenga mínimo 8 caracteres
• debería detectar si las contraseñas no coinciden
```

Nota: se ve el resultado de las pruebas de registrar Fuente: propia (2026)

Figura 9*Pruebas unitarias tareas*

```
TareasComponent
• debería iniciar mostrando las tareas pendientes
• debería crear el componente correctamente
• debería cambiar el estado de una tarea y sus estilos (a completado)
• debería cambiar el estado de una tarea y sus estilos (a proceso)
• debería mostrar un arreglo vacío si no hay tareas en un estado
• debería filtrar las tareas por estado (ej: proceso)
```

Nota: se ve el resultado de las pruebas de tareas Fuente: propia (2026)

Figura 10*Pruebas unitarias finanzas*

```
FinanzasComponent
• debería iniciar con los balances financieros cargados
• debería mostrar alertas y mensajes de consola en los métodos de interacción
• debería contener los datos dinámicos básicos (4 categorías y 2 metas)
• debería crear el componente correctamente
```

Nota: se ve el resultado de las pruebas de finanzas Fuente: propia (2026)

Figura 11*Pruebas unitarias menú principal*

```
MenuPrincipalComponent
• debería procesar un pago pendiente correctamente
• debería crear el componente correctamente
• no debería alterar un pago que ya está pagado
• debería marcar una tarea pendiente como completada
• debería configurar las fechas iniciales en el ngOnInit
• debería simular el guardado de un Gasto
• debería simular la invitación de un Miembro
• debería abrir y cerrar modales manipulando el scroll
• debería simular el guardado de una nueva Tarea
• debería navegar al perfil correctamente
```

Nota: se ve el resultado de las pruebas de menú principal Fuente: propia (2026)